

# CampusHub 校园互助服务平台架构设计文档

---

## 1. 项目概述

CampusHub 是一个面向校园场景的互助服务平台，支持用户发布任务、接单执行、订单流转、评价反馈与消息通知等核心能力。系统采用前后端分离架构，前端负责交互展示与状态管理，后端负责业务处理、数据持久化与接口服务。

---

## 2. 总体架构设计

### 2.1 架构风格

系统采用"前后端分离 + 模块化单体"的设计方式：

- 前端通过 Web 页面提供用户交互能力
- 后端通过 RESTful API 提供业务服务
- 后端内部按业务域拆分为多个模块，便于分工、维护与扩展
- 系统采用 JWT 实现无状态认证
- 通过 Redis 支撑缓存、会话控制与并发场景处理

### 2.2 技术选型

#### 前端

- Vue 3
- Vite
- TypeScript
- Element Plus
- Pinia
- Axios

#### 后端

- Spring Boot 3
- MyBatis-Plus
- Redis
- PostgreSQL
- JWT
- Redisson

### 2.3 系统交互方式

- 前端通过 Axios 调用后端 REST API
- 后端统一返回标准 JSON 响应
- 登录后由后端签发 JWT，前端保存 Token 并在请求头中携带
- 开发环境通过 Vite 代理转发 /api 到后端 8080 端口，生产环境通过环境变量切换接口地址

- 对于实时通知类功能，可预留 WebSocket 支持

---

## 3. 前端架构设计

### 3.1 设计目标

前端架构需要满足以下要求：

- 页面响应式，适配常见桌面端浏览场景
- 组件化开发，便于复用与维护
- 状态集中管理，减少跨组件传值复杂度
- 请求层统一封装，便于鉴权和错误处理
- 页面结构与后端模块边界保持一致，降低联调成本
- 支持普通用户与管理员双角色入口
- 支持实名认证、论坛互动、公告管理等业务能力扩展
- 首页采用贴吧式信息流展示任务内容，强化校园互助社区感
- 高频入口集中在顶部工具栏和左侧导航，减少跳转层级
- 个人主页以聚合视图统一承载我的足迹、收藏、发单、接单和信用信息

### 3.2 技术架构

前端采用 Vue 3 + Vite 的单页应用架构，整体分为五层：

#### 页面视图层

负责页面展示与交互组织，按业务域划分页面：

- 认证页：登录、注册
- 首页信息流页：贴吧式任务流、搜索、排序、快捷发布
- 分类页：快递代取、学习辅导、二手交易、活动组队、其他
- 发布页：按分类拆分的发布任务页
- 订单页：订单详情、我的发单、我的接单、历史订单、订单聊天
- 用户页：个人主页、资料维护、实名认证、账户注销
- 消息页：私信会话列表、聊天窗口、系统消息、公告
- 论坛页：帖子列表、帖子详情、发帖页
- 管理页：用户管理、任务管理、帖子管理、统计面板

#### 布局层

负责统一页面框架与导航结构：

- PublicLayout：登录注册等公开页面布局
- MainLayout：普通用户主站布局，包含顶部工具栏、左侧导航栏、中心信息流区域
- AdminLayout：后台管理布局，包含统计、用户、任务、论坛管理菜单

其中 MainLayout 采用三段式结构：

- 顶部栏：左侧 CampusHub 标志，中间搜索框，右侧发布按钮、消息入口、用户头像区
- 左侧栏：首页、我的、分类导航

- 中心内容区：任务信息流、分类列表、详情页或个人主页内容区

布局目录 src/layouts 包含：

- PublicLayout.vue：登录、注册页布局
- MainLayout.vue：主站三栏布局
- AdminLayout.vue：后台管理布局

## 组件层

封装可复用的通用和业务组件：

- 通用组件：分页器、搜索栏、空状态、上传组件、确认弹窗、下拉菜单
- 业务组件：任务信息流卡片、订单状态标签、用户信用徽章、帖子卡片、消息列表项、公告面板、热帖排行面板
- 交互组件：用户头像菜单、发布分类下拉框、左侧分类导航、排序切换器
- 表单组件：登录表单、实名认证表单、发帖表单、任务发布表单、评价表单

业务组件层重点包括以下复用组件：

- TaskCard：首页与分类页任务卡片
- UserAvatarMenu：顶部头像菜单
- CategoryNav：左侧分类导航
- OrderStatusBadge：订单状态标签
- EmptyState 与 LoadingBlock：列表空态与加载态组件

## 状态管理层

使用 Pinia 统一管理跨页面共享状态：

- authStore：Token、登录状态、角色、登录登出动作
- userStore：当前用户资料、信用分、实名认证状态
- appStore：全局 loading、全局消息提示、公告弹窗状态
- messageStore：未读消息数、公告列表、最近聊天会话摘要
- feedStore：当前分类、搜索关键词、排序方式、首页信息流数据

## 请求层

使用 Axios 统一封装接口访问：

- request.ts：统一创建 Axios 实例
- 请求拦截器：自动注入 Authorization
- 响应拦截器：统一处理 code、message、401 跳转和错误提示
- api 模块：按后端业务模块拆分 user、task、order、review、message、forum、admin

request.ts 统一提供以下能力：

- baseUrl 通过 VITE\_API\_BASE\_URL 读取，默认回落到 /api
- Vite 代理前缀与目标服务通过 VITE\_API\_PREFIX、VITE\_PROXY\_TARGET 控制
- 401 响应会统一清空 authStore 与 userStore，并保留 redirect 参数跳转登录页
- 业务错误和 HTTP 错误统一由请求层抛出，页面只处理成功态和少量业务分支

### 3.3 页面与后端模块映射

前端页面按后端模块进行一一映射，保证职责对齐。

| 前端页面模块    | 对应后端模块                       | 主要接口范围                   |
|-----------|------------------------------|--------------------------|
| 登录/注册     | user                         | 注册、登录                    |
| 顶部头像区     | user                         | 登录态恢复、个人主页、资料、注销         |
| 首页信息流/分类页 | task                         | 列表、详情、分类筛选、关键词搜索、排序、收藏   |
| 分类发布入口    | task                         | 发布任务                     |
| 个人主页      | user + task + order + review | 资料、信用分、主页、我的发单、我的接单、评价   |
| 我的足迹      | task + order + forum         | 浏览足迹、发单记录、接单记录、互动记录的聚合视图 |
| 我的收藏      | task + forum                 | 任务收藏、帖子收藏                |
| 私信/消息中心   | message                      | 站内消息、公告、聊天               |
| 论坛中心      | forum                        | 帖子列表、详情、评论、点赞、收藏         |
| 后台管理台     | bootstrap + admin 接口         | 用户封禁、任务下架、帖子置顶推荐、统计      |

这种映射方式有两个作用：

- 页面边界与后端接口边界一致，便于多人分工
- 前端 api 目录、router 配置和 views 目录可以保持同构，降低维护复杂度

### 3.4 路由设计

路由划分为三类：

- 公共路由：/login、/register
- 用户路由：  
由：/、/category/:category、/publish/:category、/tasks/:id、/orders、/messages、/forum、/me
- 后台路由：/admin/users、/admin/tasks、/admin/posts、/admin/stats

页面级子路由包括：

- /me/profile：个人资料
- /me/tracks：我的足迹
- /me/favorites：我的收藏
- /me/tasks：我的发单
- /me/orders：我的接单
- /me/credit：信用分与等级
- /me/account：账户注销

路由守卫规则如下：

- 未登录用户访问用户路由时跳转到登录页
- 非管理员访问后台路由时跳转到首页或 403 页面
- 登录成功后根据角色决定默认首页

具体规则如下：

- 首次导航时通过 `authStore.restoreSession` 恢复本地登录态
- `guestOnly` 页面在已登录状态下会自动跳回普通首页或后台首页
- `requiresAuth` 页面未登录时会跳转到登录页，并附带 `redirect` 参数
- `requiresAdmin` 页面会跳转到 403 页面
- 页面标题通过 `route.meta.title` 动态更新

## 3.5 状态管理设计

### 认证状态

- Token 持久化在 `localStorage`
- 登录成功后同步写入 `authStore`
- 页面初始化时恢复登录态
- Token 失效时统一清空状态并跳转登录页
- 登录页提交后会继续拉取 `getUserProfile`，补全 `userStore` 中的用户资料

### 用户状态

- 当前登录用户基础信息保存在 `userStore`
- 个人主页数据按页面进入时拉取，避免全局冗余
- 实名认证状态单独维护，便于页面级权限控制
- 用户头像区需要额外维护登录入口与头像菜单切换状态

### 消息状态

- 未读消息数在全局导航栏展示
- 登录后可主动拉取一次未读数
- 页面操作后支持局部刷新，不强依赖全量刷新

### 信息流状态

- 首页信息流按 `category + keyword + sort` 组合维护查询条件
- 排序支持按时间和按热度切换
- 分类页与首页复用同一份信息流状态模型，避免重复实现
- 首页与分类页通过 `getTaskList` 获取任务列表，并复用同一视图和状态模型

## 3.6 请求与错误处理设计

开发环境下，通过 Vite 代理解决跨域问题：

- 前端请求统一以 `/api` 开头
- Vite 将 `/api` 转发到后端服务
- 后端开启 CORS，允许前端域名访问

- 环境变量文件通过 .env.development 管理本地代理目标

请求流程如下：

1. 页面触发用户操作
2. 组件调用对应 api 模块方法
3. request.ts 自动注入 JWT Token
4. 后端返回 ApiResponse
5. 响应拦截器统一判断 code 是否为 200
6. 成功时返回 data，失败时统一弹出错误信息
7. 若遇到 401，则清理登录信息、保留当前路径并跳转登录页

关键业务链路如下：

- 登录页：校验表单 -> 调用 login -> 写入 authStore -> 调用 getUserProfile -> 跳转 redirect 或首页
- 发布页：校验表单 -> 从 authStore 注入 publisherId -> 调用 createTask -> 返回首页
- 首页：根据 category、keyword、sort 组合参数调用 getTaskList，并在 feedStore 中维护记录集

### 3.7 页面交互协同设计

#### 顶部工具栏交互

- 左上角展示 CampusHub 标志，点击返回首页并刷新信息流
- 顶部中间为全局搜索栏，用于按关键词过滤首页和分类页任务流
- 右上角发布按钮使用“+”图标，点击下拉展示分类：快递代取、学习辅导、二手交易、活动组队、其他
- 选择具体分类后直接跳转到对应发布页面
- “+”右侧为消息图标，点击进入私信/消息界面
- 未登录时右上角展示默认头像和“注册/登录”入口
- 登录后显示用户真实头像，并隐藏“注册/登录”文字入口
- 点击头像进入个人主页

#### 任务流

- 首页与分类页均采用贴吧式纵向信息流布局，列表项按“头像 + 昵称 + 信用称号 + 标题 + 描述 + 图片”展示
- 信息流支持按时间/按热度排序
- 可选增加右侧热帖排行区域，但定义为增强项，不作为首版必做依赖
- 任务详情页展示任务内容、发布者信息、收藏按钮和抢单入口
- 发布任务页与编辑任务页复用同一任务表单组件

#### 订单流

- 订单页按 role + status 分类切换列表
- 订单详情页延续信息流卡片样式，展示头像、昵称、信用称号、标题、描述与图片
- 订单详情页集中展示任务信息、状态流转按钮、评价入口与聊天入口
- 聊天区作为订单详情页子模块，避免独立页面带来的上下文切换成本

#### 用户流

- 未登录时头像区提供注册/登录入口，登录后切换为头像菜单入口
- 个人主页以标签页或侧边子菜单方式组织：昵称、头像、我的足迹、我的收藏、我的发单、我的接单、信用分与等级、账户注销
- “我的足迹”作为前端聚合视图，可由我的发单、我的接单、收藏记录与互动记录组合展示
- 个人主页展示用户公开资料、历史发布、历史接单和信用等级

## 论坛流

- 帖子列表页支持分类切换、分页和推荐位展示
- 帖子详情页展示正文、评论、点赞收藏操作
- 发帖页支持文本与图片链接输入

## 管理流

- 管理端首页展示统计面板
- 用户管理页调用封禁与认证审核接口
- 任务管理页调用下架接口
- 帖子管理页调用置顶和推荐接口

## 3.8 前端目录设计

```
src/
├── api/                # 接口封装
│   ├── request.ts     # Axios 实例、鉴权与错误处理
│   ├── user.ts        # 用户与认证接口
│   ├── task.ts        # 任务接口
│   ├── order.ts       # 订单接口
│   ├── review.ts      # 评价接口
│   ├── message.ts     # 消息与公告接口
│   ├── forum.ts       # 论坛接口
│   └── admin.ts       # 后台管理接口
├── assets/            # 静态资源
├── components/        # 公共组件与业务组件预留目录
├── constants/         # 常量与导航配置
├── layouts/           # 页面布局
│   ├── PublicLayout.vue
│   ├── MainLayout.vue
│   └── AdminLayout.vue
├── router/            # 路由配置与守卫
├── stores/            # Pinia 状态管理
│   ├── index.ts       # Pinia 实例导出
│   ├── auth.ts
│   ├── user.ts
│   ├── app.ts
│   ├── message.ts
│   └── feed.ts
├── types/             # TypeScript 类型定义
├── utils/             # Token 与通用工具
├── views/             # 页面视图
│   └── admin/
```

```
|   |   |— auth/
|   |   |— forum/
|   |   |— home/
|   |   |— message/
|   |   |— order/
|   |   |— profile/
|   |   |— publish/
|   |   |— system/
|   |   |— task/
|— App.vue           # 根组件
|— main.ts           # 入口文件
|— style.css         # 全局样式
|— vite-env.d.ts     # Vite 环境变量类型声明
```

## 4. 后端架构设计

### 4.1 设计目标

后端架构需要满足：

- 业务边界清晰
- 模块职责单一
- 支持高并发场景
- 易于扩展和测试
- 保持较低耦合度

### 4.2 架构模式

后端采用 Maven 多模块模块化单体架构，统一由父工程管理依赖和版本。

### 4.3 模块划分

```
campushub-parent/
|— campushub-common/    # 公共模块
|— campushub-user/      # 用户模块
|— campushub-task/      # 任务模块
|— campushub-order/     # 订单模块
|— campushub-review/    # 评价模块
|— campushub-message/   # 消息模块
|— campushub-forum/     # 论坛模块
|— campushub-bootstrap/ # 启动模块
```

### 4.4 各模块职责

#### 4.4.1 common 模块

提供通用能力，包括：



- 全局异常处理
- 统一返回体
- JWT 工具类
- 公共常量
- CORS 配置
- 通用枚举与工具类

#### 4.4.2 user 模块

负责用户相关业务，包括：

- 用户注册与登录
- 实名认证与认证审核
- 用户资料维护
- 账号注销与注销校验
- 角色权限管理
- 信用分维护
- 个人主页与信用等级展示

#### 4.4.3 task 模块

负责任务发布与浏览，包括：

- 任务发布
- 任务编辑
- 任务列表查询
- 分类筛选
- 关键词搜索
- 任务收藏
- 地理位置标记
- 任务状态管理

#### 4.4.4 order 模块

负责接单与订单流转，包括：

- 抢单
- 接单确认
- 订单取消
- 订单状态变更
- 订单完成确认
- 历史订单分类查询
- 并发控制

#### 4.4.5 review 模块

负责评价与信誉管理，包括：

- 双向评价

- 评分记录
- 信誉分计算
- 评价展示

#### 4.4.6 message 模块

负责站内消息与通知，包括：

- 系统通知与系统公告
- 订单状态通知
- 基于订单的简易聊天
- 评价通知
- 未读消息管理
- 可扩展 WebSocket 推送

#### 4.4.7 forum 模块

负责论坛帖子与互动能力，包括：

- 帖子发布
- 帖子编辑与删除
- 帖子列表查询
- 帖子点赞与收藏
- 评论与回复入口
- 置顶与推荐等后台管理能力

#### 4.4.8 bootstrap 模块

作为应用启动入口，负责：

- Spring Boot 启动类
- 配置加载
- 模块装配
- 后台统计聚合接口装配

### 4.5 各模块详细架构设计

#### 4.5.1 common 模块架构

**模块职责：**提供全系统通用的基础设施和工具支持。

**分层设计：**

```
common/
├── config/           # 全局配置
│   ├── CorsConfig   # CORS 跨域配置
│   ├── JwtConfig     # JWT 配置
│   └── RedisConfig   # Redis 配置
└── audit/           # 内容审核
```

```

├── SensitiveWordFilter # 敏感词过滤器
├── ContentAuditService # 内容审核服务
├── exception/          # 异常处理
│   ├── GlobalExceptionHandler # 全局异常拦截器
│   ├── BusinessException     # 业务异常
│   └── SystemException        # 系统异常
├── response/           # 统一响应
│   ├── ApiResponse        # 标准响应体
│   └── PageResponse       # 分页响应体
├── security/           # 安全相关
│   ├── JwtTokenProvider   # JWT 生成与验证
│   ├── JwtAuthFilter      # JWT 认证过滤器
│   └── SecurityUtils      # 安全工具类
├── constant/           # 常量定义
│   ├── ErrorCode          # 错误码常量
│   ├── MessageType        # 消息类型常量
│   └── OrderStatus        # 订单状态常量
├── enums/              # 枚举定义
│   ├── UserRole           # 用户角色枚举
│   ├── TaskStatus        # 任务状态枚举
│   └── OrderStatus        # 订单状态枚举
├── utils/              # 工具类
│   ├── DateUtils          # 日期工具
│   ├── EncryptUtils       # 加密工具
│   └── ValidateUtils      # 验证工具

```

#### 核心类职责：

- **ApiResponse**: 统一返回体，包含 code、message、data 三个字段
- **JwtTokenProvider**: 负责 JWT 的生成、验证和刷新
- **SensitiveWordFilter**: 为任务、帖子、评论提供敏感词过滤能力
- **GlobalExceptionHandler**: 捕获全局异常，统一返回错误响应
- **JwtAuthFilter**: 拦截请求，验证 Token 有效性

#### 对外接口：

- 无 REST 接口，仅提供工具类和配置

#### 4.5.2 user 模块架构

**模块职责：**管理用户账户、认证、资料和信用体系。

#### 分层设计：

```

user/
├── controller/          # 控制层
│   ├── AuthController  # 认证接口入口
│   ├── UserController  # 用户接口入口
│   └── AdminUserController # 后台用户管理入口
├── service/             # 业务层

```

```
|   ├── UserService      # 用户业务逻辑
|   ├── AuthService      # 认证业务逻辑
|   ├── CreditService    # 信用分业务逻辑
|   └── VerificationService # 实名认证业务逻辑
├── mapper/              # 数据访问层
|   └── UserMapper        # 用户数据操作
├── entity/              # 实体类
|   └── User              # 用户实体
├── dto/                 # 数据传输对象
|   ├── LoginRequest      # 登录请求
|   ├── LoginResponseDTO  # 登录响应
|   ├── RegisterRequest   # 注册请求
|   ├── VerificationSubmitRequest # 认证提交请求
|   ├── UserInfoDTO       # 用户信息 DTO
|   ├── UserProfileDTO    # 用户资料 DTO
|   └── UserHomeDTO       # 个人主页 DTO
└── event/               # 事件发布
    └── UserCreditEvent   # 信用分变更事件
```

#### 核心类职责：

- **UserService**: 用户资料查询与更新
- **AuthService**: 注册、登录、注销等认证入口
- **CreditService**: 信用分查询与更新入口
- **VerificationService**: 学生证上传、校内认证结果处理与认证状态维护
- **AuthController**: 暴露注册、登录 REST 接口
- **AdminUserController**: 暴露封禁与实名认证审核等后台管理接口

#### 对外接口：

- **POST /api/user/register** - 用户注册
- **POST /api/user/login** - 用户登录
- **POST /api/user/verification/submit** - 提交学生证或认证材料
- **GET /api/user/profile?userId={id}** - 获取用户资料
- **PUT /api/user/profile?userId={id}** - 更新用户资料
- **GET /api/user/credit?userId={id}** - 获取信用分
- **GET /api/user/{userId}** - 获取公开用户信息与信用等级
- **GET /api/user/{userId}/home** - 获取个人主页聚合信息（历史发布、接单记录、信用等级）
- **DELETE /api/user/account?userId={id}** - 注销账号并校验未完成订单
- **POST /api/admin/users/{userId}/ban** - 后台封禁用户
- **POST /api/admin/users/{userId}/verify** - 后台审核实名认证

#### 内部依赖：

- 依赖 common 模块的 JWT、异常处理、常量
- 通过任务与订单查询接口协同支撑个人主页展示

---

### 4.5.3 task 模块架构

**模块职责：**管理任务的发布、编辑、查询和状态流转。

**分层设计：**

```
task/
├── controller/           # 控制层
│   ├── TaskController   # 任务接口入口
│   └── AdminTaskController # 后台任务管理入口
├── service/             # 业务层
│   ├── TaskService      # 任务业务逻辑
│   ├── TaskQueryService # 任务查询逻辑
│   ├── TaskFavoriteService # 任务收藏业务逻辑
│   └── TaskStatusService # 任务状态管理
├── mapper/              # 数据访问层
│   └── TaskMapper        # 任务数据操作
├── entity/              # 实体类
│   └── Task              # 任务实体
├── dto/                  # 数据传输对象
│   ├── TaskCreateRequest # 任务创建请求
│   ├── TaskUpdateRequest # 任务更新请求
│   ├── TaskDetailDTO      # 任务详情 DTO
│   └── TaskListDTO        # 任务列表 DTO
├── event/                # 事件发布
│   └── TaskPublishedEvent # 任务发布事件
```

**核心类职责：**

- **TaskService**: 任务创建、编辑、删除、状态变更
- **TaskQueryService**: 任务列表、分类筛选、关键词搜索入口
- **TaskFavoriteService**: 任务收藏、取消收藏与收藏列表查询
- **TaskStatusService**: 删除与状态控制入口
- **TaskController**: 暴露 REST 接口

**对外接口：**

- **POST /api/tasks** - 发布任务
- **PUT /api/tasks/{taskId}** - 编辑任务
- **GET /api/tasks/{taskId}** - 获取任务详情
- **GET /api/tasks?category=&keyword=&sort=&page=1&size=10** - 获取任务列表
- **POST /api/tasks/{taskId}/favorite** - 收藏任务
- **DELETE /api/tasks/{taskId}/favorite** - 取消收藏任务
- **GET /api/tasks/favorites?userId={id}&page=1&size=10** - 获取任务收藏列表
- **DELETE /api/tasks/{taskId}** - 删除任务
- **DELETE /api/admin/tasks/{taskId}** - 后台下架任务

**内部依赖：**

- 依赖 common 模块的异常处理、常量
- 依赖 common 模块的敏感词过滤能力进行内容预审
- 发布事件供 order、message 模块监听

#### 4.5.4 order 模块架构

**模块职责：**管理订单的创建、流转、并发控制和完成确认。

**分层设计：**

```
order/
├── controller/           # 控制层
│   └── OrderController  # 订单接口入口
├── service/             # 业务层
│   ├── OrderService    # 订单业务逻辑
│   ├── GrabService     # 抢单业务逻辑（并发控制）
│   └── OrderStatusService # 订单状态管理
├── mapper/              # 数据访问层
│   └── OrderMapper      # 订单数据操作
├── entity/              # 实体类
│   └── Order            # 订单实体
├── dto/                  # 数据传输对象
│   ├── GrabOrderRequest # 抢单请求
│   ├── OrderDetailDTO   # 订单详情 DTO
│   └── OrderListDTO     # 订单列表 DTO
├── lock/                 # 并发控制
│   └── RedissonLock     # 基于 Redisson 的分布式锁
└── event/                # 事件发布
    ├── OrderCreatedEvent # 订单创建事件
    ├── OrderConfirmedEvent # 订单确认事件
    └── OrderCompletedEvent # 订单完成事件
```

**核心类职责：**

- **OrderService**: 订单详情与列表查询
- **GrabService**: 抢单逻辑，使用 Redisson 分布式锁保证并发安全
- **OrderStatusService**: 接单确认、取消、完成等状态流转管理
- **OrderController**: 暴露 REST 接口

**对外接口：**

- **POST /api/orders/grab** - 抢单（高并发）
- **POST /api/orders/{orderId}/confirm** - 确认接单
- **POST /api/orders/{orderId}/cancel** - 取消订单
- **POST /api/orders/{orderId}/complete** - 完成订单
- **GET /api/orders/{orderId}** - 获取订单详情
- **GET /api/orders?userId={id}&role=poster|helper&status=&page=1&size=10** - 获取订单列表
- **POST /api/admin/reports/{reportId}/handle** - 后台处理举报

**内部依赖：**

- 依赖 common 模块的异常处理、常量
- 依赖 task 模块的任务信息

- 依赖 user 模块的用户信息
- 订单取消事件可触发信用分扣减与消息通知
- 发布事件供 review、message 模块监听

#### 4.5.5 review 模块架构

**模块职责：**管理订单完成后的双向评价和信誉分计算。

**分层设计：**

```
review/
├── controller/           # 控制层
│   └── ReviewController # 评价接口入口
├── service/             # 业务层
│   ├── ReviewService    # 评价业务逻辑
│   └── CreditCalculator # 信誉分计算器
├── mapper/              # 数据访问层
│   └── ReviewMapper      # 评价数据操作
├── entity/              # 实体类
│   └── Review            # 评价实体
├── dto/                 # 数据传输对象
│   ├── ReviewCreateRequest # 评价创建请求
│   ├── ReviewDetailDTO     # 评价详情 DTO
│   └── ReviewListDTO       # 评价列表 DTO
└── event/               # 事件监听
    └── OrderCompletedEventListener # 监听订单完成事件
```

**核心类职责：**

- **ReviewService**: 评价创建、查询、统计
- **CreditCalculator**: 根据评价计算用户信誉分与信用等级映射输入
- **ReviewController**: 暴露 REST 接口

**对外接口：**

- **POST /api/reviews** - 创建评价
- **GET /api/reviews/{reviewId}** - 获取评价详情
- **GET /api/reviews/user/{userId}?page=1&size=10** - 获取用户评价列表

**内部依赖：**

- 依赖 common 模块的异常处理、常量
- 依赖 order 模块的订单信息
- 依赖 user 模块的用户信息
- 监听 order 模块的订单完成事件

#### 4.5.6 message 模块架构

**模块职责：**管理系统消息、订单通知、评价通知和未读消息状态。

**分层设计：**

```
message/
├── controller/           # 控制层
│   └── MessageController # 消息接口入口
├── service/             # 业务层
│   ├── MessageService   # 消息业务逻辑
│   ├── ChatService      # 订单聊天业务逻辑
│   ├── NoticeService    # 系统公告业务逻辑
│   ├── NotificationService # 通知发送逻辑
│   └── MessageQueryService # 消息查询逻辑
├── mapper/              # 数据访问层
│   └── MessageMapper     # 消息数据操作
├── entity/              # 实体类
│   └── Message           # 消息实体
├── dto/                  # 数据传输对象
│   ├── MessageDetailDTO  # 消息详情 DTO
│   ├── MessageListDTO   # 消息列表 DTO
│   ├── ChatMessageDTO    # 聊天消息 DTO
│   └── NoticeDTO         # 公告 DTO
├── event/                # 事件监听
│   ├── OrderEventListener # 监听订单事件
│   ├── ReviewEventListener # 监听评价事件
│   └── TaskEventListener  # 监听任务事件
└── websocket/            # WebSocket 支持（可选）
    └── MessageWebSocketHandler # WebSocket 消息推送
```

**核心类职责：**

- **MessageService**: 消息状态维护
- **ChatService**: 提供基于订单的一对一简易聊天与消息留存
- **NoticeService**: 提供系统公告发布、列表查询与首页展示能力
- **NotificationService**: 根据事件生成通知消息
- **MessageQueryService**: 消息列表查询、未读消息统计
- **MessageController**: 暴露 REST 接口

**对外接口：**

- GET /api/messages?userId={id}&page=1&size=10 - 获取消息列表
- GET /api/messages/unread-count?userId={id} - 获取未读消息数
- PUT /api/messages/{messageId}/read - 标记消息已读
- DELETE /api/messages/{messageId} - 删除消息
- GET /api/orders/{orderId}/chat?page=1&size=20 - 获取订单聊天记录
- POST /api/orders/{orderId}/chat - 发送聊天消息
- GET /api/notices/latest - 获取首页公告
- GET /api/admin/notices?page=1&size=10 - 获取公告管理列表
- POST /api/admin/notices - 发布系统公告



### 内部依赖：

- 依赖 common 模块的异常处理、常量
- 依赖 task、order 模块事件类型
- 监听 order、review、task 模块的事件

## 4.5.7 forum 模块架构

**模块职责：**管理论坛帖子、评论入口和后台置顶扩展点。

### 分层设计：

```
forum/
├── controller/           # 控制层
│   ├── ForumController  # 论坛接口入口
│   └── AdminForumController # 后台论坛管理入口
├── service/             # 业务层
│   ├── ForumService     # 帖子与评论业务入口
│   └── PostInteractionService # 点赞、收藏与推荐业务入口
├── mapper/              # 数据访问层
│   ├── PostMapper       # 帖子数据操作
│   └── CommentMapper     # 评论数据操作
├── entity/              # 实体类
│   ├── Post             # 帖子实体
│   └── Comment           # 评论实体
├── dto/                 # 数据传输对象
│   ├── PostCreateRequest # 帖子创建/更新请求
│   ├── PostDetailDTO     # 帖子详情 DTO
│   ├── PostListDTO       # 帖子列表 DTO
│   └── CommentCreateRequest # 评论创建请求
└── event/               # 事件发布
    └── PostPublishedEvent # 帖子发布事件
```

### 核心类职责：

- **ForumService**: 帖子与评论的统一入口
- **PostInteractionService**: 处理点赞、收藏、浏览量与推荐标记相关能力
- **ForumController**: 暴露帖子发布、列表、详情、修改、删除、评论接口
- **AdminForumController**: 暴露帖子置顶、推荐等后台管理接口

### 对外接口：

- **POST /api/posts** - 发布帖子
- **GET /api/posts/{postId}** - 获取帖子详情
- **GET /api/posts?category=&sort=latest|views|recommend&page=1&size=10** - 获取帖子列表
- **PUT /api/posts/{postId}** - 更新帖子
- **DELETE /api/posts/{postId}** - 删除帖子
- **POST /api/posts/{postId}/comments** - 发表评论
- **POST /api/posts/{postId}/like** - 点赞帖子

- DELETE /api/posts/{postId}/like - 取消点赞帖子
- POST /api/posts/{postId}/favorite - 收藏帖子
- DELETE /api/posts/{postId}/favorite - 取消收藏帖子
- POST /api/admin/posts/{postId}/pin - 后台置顶帖子
- POST /api/admin/posts/{postId}/recommend - 后台推荐帖子

#### 内部依赖:

- 依赖 common 模块的异常处理、常量
- 依赖 common 模块的敏感词过滤能力进行发帖与评论预审
- 依赖 user 模块的用户信息作为扩展点

---

### 4.5.8 bootstrap 模块架构

**模块职责:** 应用启动入口，负责模块装配和配置加载。

#### 分层设计:

```
bootstrap/
├── CampusHubApplication.java # Spring Boot 启动类
├── controller/
│   └── AdminStatsController # 后台统计聚合接口
├── service/
│   └── StatsQueryService # 统计聚合服务
└── resources/
    ├── application.yml # 基础应用配置
    └── db/
        └── schema.sql # 统一建表脚本
```

#### 核心职责:

- 启动 Spring Boot 应用
- 加载所有子模块
- 承载统一配置入口
- 为后台提供跨模块统计聚合接口

#### 对外接口:

- GET /api/admin/stats/dashboard - 获取后台聚合统计面板

---

## 4.6 模块间通信方式

#### 模块依赖关系

- campushub-common 被所有业务模块依赖
- campushub-user 提供用户基础信息与认证入口
- campushub-task 仅依赖 common
- campushub-order 依赖 task、user

- `campushub-review` 依赖 `order`、`user`
- `campushub-message` 依赖 `task`、`order` 的事件类型
- `campushub-forum` 依赖 `common`、`user`
- `campushub-bootstrap` 聚合所有模块并提供启动入口

## 同步调用

模块间通过内部服务接口进行调用，不直接跨模块访问 Mapper，避免业务层耦合过深。

## 异步通信

通过事件机制实现模块解耦。例如：

- 任务发布后发布 `TaskPublishedEvent`
- 订单创建后发布 `OrderCreatedEvent`
- 订单完成后发布 `OrderCompletedEvent`
- 用户模块监听事件并增加信用分
- 消息模块监听事件并发送通知

## 4.7 核心业务设计

### 抢单并发控制

抢单属于高并发场景，需要采用分布式锁控制资源竞争：

- 以订单 ID 或任务 ID 作为锁粒度
- 获取锁后再进行状态校验与创建订单
- 业务完成后释放锁
- 配合乐观锁防止重复更新

### 状态机控制

订单状态应严格按流程流转，例如：

- `PENDING`
- `CONFIRMED`
- `COMPLETED`
- `CANCELLED`

状态变更应由服务层统一控制，避免前端直接传入任意状态。

### 取消与历史订单规则

- 订单在 `PENDING` 或 `CONFIRMED` 阶段允许取消，但取消方与取消时点会影响信用分扣减策略
- 历史订单通过 `role + status` 组合查询，支撑“进行中 / 已完成 / 已取消”分类展示
- 订单取消、完成、举报处理均可触发信用分更新与消息通知事件

---

## 5. 数据库设计

## 5.1 数据库选型

采用 PostgreSQL 作为主数据库，支持事务、并发控制与复杂查询场景。

## 5.2 设计原则

- 表结构按业务域划分
- 主键统一使用自增或雪花 ID
- 核心业务表保留创建时间、更新时间、逻辑删除字段
- 高频更新字段增加索引
- 订单等核心状态表保留版本号字段支持乐观锁

## 5.3 核心数据表

### 5.3.1 用户表 `u_user`

用途：存储用户基础信息与账户状态。

主要字段：

- `id` - 主键
- `username` - 用户名
- `password` - 密码（加密存储）
- `email` - 邮箱
- `phone` - 电话
- `nickname` - 昵称
- `real_name` - 真实姓名
- `department` - 所属院系
- `avatar` - 头像 URL
- `role` - 角色
- `credit_score` - 信用分
- `status` - 账户状态
- `deleted` - 逻辑删除标识
- `created_at` - 创建时间
- `updated_at` - 更新时间

说明：

- `role` 用于区分普通用户、管理员等角色
- `credit_score` 用于记录用户信用分
- `credit_level` 不单独存储，由 `credit_score` 按规则映射生成
- `deleted` 用于逻辑删除

### 5.3.2 任务表 `t_task`

用途：存储用户发布的任务信息。

主要字段：

- `id` - 主键

- `publisher_id` - 发布者 ID
- `title` - 任务标题
- `description` - 任务描述
- `category` - 分类
- `reward` - 报酬
- `location` - 地理位置
- `status` - 任务状态
- `deadline` - 截止时间
- `created_at` - 创建时间
- `updated_at` - 更新时间
- `deleted` - 逻辑删除标识

说明：

- `publisher_id` 关联用户表
- `location` 预留用于地理位置扩展
- `status` 表示任务当前状态

### 5.3.3 订单表 `t_order`

用途：存储任务接单后的订单流转信息。

主要字段：

- `id` - 主键
- `task_id` - 任务 ID
- `poster_id` - 发布者 ID
- `helper_id` - 接单者 ID
- `status` - 订单状态
- `version` - 版本号（乐观锁）
- `created_at` - 创建时间
- `updated_at` - 更新时间
- `deleted` - 逻辑删除标识

说明：

- `version` 用于乐观锁
- `status` 用于订单状态控制
- `task_id` 关联任务表

### 5.3.4 评价表 `t_review`

用途：存储订单完成后的双向评价记录。

主要字段：

- `id` - 主键
- `order_id` - 订单 ID
- `reviewer_id` - 评价者 ID
- `target_user_id` - 被评价者 ID

- `score` - 评分
- `comment` - 评价内容
- `created_at` - 创建时间
- `deleted` - 逻辑删除标识

说明：

- 支持双方互评
- `score` 用于信誉计算

### 5.3.5 消息表 `t_message`

用途：存储站内信和系统通知。

主要字段：

- `id` - 主键
- `receiver_id` - 接收者 ID
- `type` - 消息类型
- `title` - 消息标题
- `content` - 消息内容
- `read` - 已读状态
- `created_at` - 创建时间
- `deleted` - 逻辑删除标识

说明：

- `read` 表示是否已读
- `type` 区分系统消息、订单消息、评价消息等

### 5.3.6 论坛帖子表 `t_post`

用途：存储论坛帖子主体信息。

主要字段：

- `id` - 主键
- `author_id` - 作者 ID
- `category` - 帖子分类
- `title` - 帖子标题
- `content` - 帖子内容
- `media_urls` - 图片资源列表
- `view_count` - 浏览量
- `like_count` - 点赞数
- `favorite_count` - 收藏数
- `is_pinned` - 是否置顶
- `is_recommended` - 是否推荐
- `created_at` - 创建时间
- `updated_at` - 更新时间
- `deleted` - 逻辑删除标识

说明：

- `author_id` 关联用户表
- `media_urls` 支撑图文帖需求
- `is_pinned` 与 `is_recommended` 支撑后台运营能力

### 5.3.7 论坛评论表 `t_comment`

用途：存储帖子评论及回复关系。

主要字段：

- `id` - 主键
- `post_id` - 所属帖子 ID
- `author_id` - 评论作者 ID
- `reply_to_comment_id` - 被回复评论 ID
- `content` - 评论内容
- `created_at` - 创建时间
- `deleted` - 逻辑删除标识

说明：

- 通过 `reply_to_comment_id` 支持回复链路
- 通过评论实体与数据访问层支撑评论与回复链路

### 5.3.8 用户认证表 `t_user_verification`

用途：存储学生证上传、统一身份认证及审核结果。

主要字段：

- `id` - 主键
- `user_id` - 用户 ID
- `auth_type` - 认证方式（学生证/统一身份认证）
- `student_no` - 学号
- `student_card_url` - 学生证材料地址
- `auth_status` - 认证状态
- `reviewer_id` - 审核管理员 ID
- `submitted_at` - 提交时间
- `verified_at` - 通过时间

说明：

- `auth_status` 支撑实名认证流程
- 用户只有通过认证后才能发布论坛帖子或接入更多核心能力

### 5.3.9 任务收藏表 `t_task_favorite`

用途：存储用户对互助任务的收藏关系。

主要字段：

- `id` - 主键
- `user_id` - 用户 ID
- `task_id` - 任务 ID
- `created_at` - 创建时间

说明：

- 用于支撑 FR-08 需求收藏功能
- `(user_id, task_id)` 应保持唯一

#### 5.3.10 订单聊天消息表 `t_chat_message`

用途：存储基于订单的一对一简易聊天记录。

主要字段：

- `id` - 主键
- `order_id` - 订单 ID
- `sender_id` - 发送者 ID
- `receiver_id` - 接收者 ID
- `content` - 聊天内容
- `created_at` - 创建时间
- `deleted` - 逻辑删除标识

说明：

- 聊天消息与订单绑定，便于上下文追踪
- 仅支持文本消息，满足课程项目范围

#### 5.3.11 系统公告表 `t_notice`

用途：存储首页展示和后台管理的系统公告。

主要字段：

- `id` - 主键
- `title` - 公告标题
- `content` - 公告内容
- `status` - 公告状态
- `publisher_id` - 发布管理员 ID
- `created_at` - 创建时间
- `expired_at` - 失效时间

说明：

- 公告与普通站内消息分离，便于首页聚合展示

#### 5.3.12 帖子点赞表 `t_post_like`

用途：存储用户对论坛帖子的点赞关系。



主要字段：

- `id` - 主键
- `user_id` - 用户 ID
- `post_id` - 帖子 ID
- `created_at` - 创建时间

说明：

- 用于支撑帖子点赞与取消点赞
- `(user_id, post_id)` 应保持唯一

### 5.3.13 帖子收藏表 `t_post_favorite`

用途：存储用户对论坛帖子的收藏关系。

主要字段：

- `id` - 主键
- `user_id` - 用户 ID
- `post_id` - 帖子 ID
- `created_at` - 创建时间

说明：

- 用于支撑帖子收藏与取消收藏
- `(user_id, post_id)` 应保持唯一

## 5.4 关系设计

- `u_user` 与 `t_task` 为一对多
- `u_user` 与 `t_order` 为一对多
- `t_task` 与 `t_order` 为一对一或一对多，取决于业务是否允许多次抢单
- `t_order` 与 `t_review` 为一对多或一对二
- `u_user` 与 `t_message` 为一对多
- `u_user` 与 `t_user_verification` 为一对多
- `u_user` 与 `t_task_favorite` 为一对多
- `t_order` 与 `t_chat_message` 为一对多
- `u_user` 与 `t_notice` 为一对多（管理员发布）
- `u_user` 与 `t_post` 为一对多
- `t_post` 与 `t_comment` 为一对多
- `u_user` 与 `t_post_like` 为一对多
- `u_user` 与 `t_post_favorite` 为一对多
- `t_post` 与 `t_post_like` 为一对多
- `t_post` 与 `t_post_favorite` 为一对多

## 5.5 索引设计

建立以下索引：

- `u_user.username`

- u\_user.phone
  - u\_user.department
  - t\_task.status
  - t\_task.category
  - t\_task.location
  - t\_order.task\_id
  - t\_order.helper\_id
  - t\_order.status
  - t\_message.receiver\_id
  - t\_message.read
  - t\_user\_verification.user\_id
  - t\_user\_verification.auth\_status
  - t\_task\_favorite.user\_id
  - t\_task\_favorite.task\_id
  - t\_chat\_message.order\_id
  - t\_notice.status
  - t\_post.author\_id
  - t\_post.category
  - t\_post.view\_count
  - t\_post.is\_pinned
  - t\_post.is\_recommended
  - t\_comment.post\_id
  - t\_post\_like.user\_id
  - t\_post\_favorite.user\_id
-